

```

GPIO.output(led, GPIO.HIGH)
print("ON")
time.sleep(1)
#turn off, set as LOW or 0
GPIO.output(led, GPIO.LOW)
print("OFF")
time.sleep(1)
except KeyboardInterrupt:
    #cleanup GPIO settings before exiting
    GPIO.cleanup()
    print("Exiting...")

```

This code will print *ON* and *OFF* alternatively on the screen, in sync with when the LED is turned on and off. The *Ctrl+C* key combination can be used to terminate the execution of the program. The *except KeyboardInterrupt*: mechanism is used to detect the *Ctrl+C* keypress. The *Sleep* method will make the process wait for the given amount of time, which is one second here.

Setting up an input pin

Just like a ground pin was used to complete the circuit of the output pin, the circuit of an input pin should be completed using a ground pin or 3v3 pin. A lone input pin in a circuit is said to be 'floating'. Since its voltage can be of any value between 0 and 3.3V, it cannot be used. That should be avoided by using a 3v3 pin or a ground pin and an in-built pull up or pull down resistor.

A pin can be set as input as follows:

```
GPIO.setup(channel, GPIO.IN, PUD)
```

Here, the channel is the pin number. *PUD* can be either *GPIO.PUD_DOWN* or *GPIO.PUD_UP*. It tells you whether to use the inbuilt pull up or the pull down resistor. If the 3v3 pin is used, we have to use *GPIO.PUD_DOWN* (pull down resistor), and for the ground pin, we need to use *GPIO.PUD_UP* (pull up resistor). The choice of 3v3 or ground is up to you. The only difference is the value read from an open circuit and closed circuit. For the 3v3 pin (*GPIO.PUD_DOWN*), the input value of the open circuit will be 0 and of a closed circuit will be 1. For a ground pin (*GPIO.PUD_UP*), open circuit is 1 and closed circuit is 0.

The value can be read from an input pin using the method *GPIO.input(channel)*, usually read into a variable.

Practice circuit 2

Fix a two-legged button switch on the breadboard. Now, connect one leg of the switch to any GPIO pin, say Pin 7, and connect the other to any 3.3V pin.

```

#!/usr/bin/python3
import RPi.GPIO as GPIO
import time

```

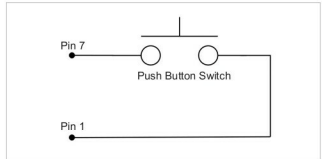


Figure 3: Circuit 2

```

cnl = 7
GPIO.setmode(GPIO.BOARD)

# PIN 7 AND 3.3V
# normally 0 when connected 1
GPIO.setup(cnl, GPIO.IN, GPIO.PUD_DOWN)
try:
    while(True):
        print(GPIO.input(cnl))
        time.sleep(1)
except KeyboardInterrupt:
    GPIO.cleanup()
    print("Exiting")

```

When this program is run, it will print 0 continuously, and when the switch is pressed down (when the circuit is closed) the output will be 1.

Now change the circuit a little. Remove the connection from the 3v3 pin and connect it to any ground pin. Change the program by replacing *GPIO.PUD_DOWN* with *GPIO.PUD_UP*. Now run the program, and 1 will be printed continuously. When the switch is pressed down, the output will be 0.

Interrupts

There is a problem with simple input pins, which is that the programmer has to constantly check the input data. To overcome this, we have interrupts known as 'event-detect' and a pin can be made to listen to events by calling:

```
GPIO.add_event_detect(channel, event, callback = my_callback,
bouncetime = timeinmilliseconds)
```

Channel should be an input pin. The various events available are *GPIO.RISING* for the rising edge (when the signal changes to HIGH from LOW), *GPIO.FALLING* for the falling edge (when the signal changes from HIGH to LOW), and *GPIO.BOTH* for both rising and falling edges. The *my_callback* method contains the interrupt handling code, and will be executed as a separate thread. *bouncetime* is the minimum amount of time required between two events. If two events occur in succession, within the *bouncetime* limit, then the second event will be ignored.